

Day 18 — Dutch National Flag Algorithm

Three-Way Partitioning • In-Place Array Algorithm

FROM	TO
Unsorted 0 / 1 / 2 array	0s → 1s → 2s

Goal: Sort an array containing only 0, 1 and 2 in one pass, without using an extra array.

GitHub: github.com/ritesh2687/dsa_ppt_post

1. ■ What is it?

The Dutch National Flag (DNF) Algorithm is an in-place partitioning technique introduced by Edsger W. Dijkstra. It is commonly used to sort an array containing exactly three categories: 0, 1 and 2.

Instead of comparing every element with every other element, the algorithm maintains three regions and places each value into its correct region while scanning the array once.

2. ■ Why do we need it?

A simple approach is to sort the array using a general-purpose sorting algorithm, but that can take $O(n \log n)$. Counting frequencies can achieve $O(n)$, but needs an additional pass.

DNF is useful when:

- The array contains three known categories.
- We want $O(n)$ time.
- We want $O(1)$ extra space.
- We want an in-place solution.

3. ■■■ Core Idea

Maintain three pointers:

- low — boundary after the region of 0s.
- mid — current element being examined.
- high — boundary before the region of 2s.

Invariant:

$[0 \dots \text{low}-1] \rightarrow \text{all } 0$

$[\text{low} \dots \text{mid}-1] \rightarrow \text{all } 1$

$[\text{mid} \dots \text{high}] \rightarrow \text{unknown}$

$[\text{high}+1 \dots n-1] \rightarrow \text{all } 2$

The algorithm repeatedly examines `nums[mid]`.

4. ■■ How does it work?

Case 1 — `nums[mid] == 0`

Swap `nums[low]` and `nums[mid]`. Increment both `low` and `mid`.

Case 2 — `nums[mid] == 1`

It is already in the middle region. Only increment `mid`.

Case 3 — `nums[mid] == 2`

Swap `nums[mid]` and `nums[high]`. Decrement `high`. Do NOT increment `mid` because the newly swapped element has not been examined yet.

5. ■ Step-by-Step Example

Input: [2, 0, 2, 1, 1, 0]

Start:

low = 0, mid = 0, high = 5

mid sees 2 → swap with high

[0, 0, 2, 1, 1, 2]

high = 4

mid sees 0 → swap with low

[0, 0, 2, 1, 1, 2]

low = 1, mid = 1

mid sees 0 → already correct

low = 2, mid = 2

mid sees 2 → swap with high

[0, 0, 1, 1, 2, 2]

high = 3

mid sees 1 → move mid

mid = 3

mid sees 1 → move mid

mid = 4

Stop because mid > high.

Output: [0, 0, 1, 1, 2, 2]

6. ■ C++ Implementation

The implementation below performs the complete partition in one scan.

```
#include <bits/stdc++.h>
using namespace std;

void sortColors(vector& nums) {
    int low = 0, mid = 0;
    int high = nums.size() - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums[low], nums[mid]);
            low++;
            mid++;
        }
        else if (nums[mid] == 1) {
            mid++;
        }
        else { // nums[mid] == 2
            swap(nums[mid], nums[high]);
            high--;
        }
    }
}

int main() {
    vector nums = {2, 0, 2, 1, 1, 0};

    sortColors(nums);

    for (int x : nums)
        cout << x << " ";

    return 0;
}
```

7. ■ Dry Run Table

For [2, 0, 2, 1, 1, 0]:

At each step, only the unknown region needs attention.

mid value	Action	Resulting idea
---	---	---
2	swap with high	Move 2 to the end
0	swap with low	Move 0 to the front
0	move boundaries	0 region grows
2	swap with high	2 region grows
1	mid++	1 stays in middle
1	mid++	1 stays in middle

Final: [0, 0, 1, 1, 2, 2]

8. ■ The Three Regions

Think of the array as four sections:

0-region | 1-region | UNKNOWN | 2-region

Initially:

UNKNOWN occupies the whole array.

During execution:

- low expands the 0-region.
- mid shrinks the unknown region.
- high expands the 2-region.

When mid crosses high, no unknown elements remain, so the array is sorted.

9. ■■ Time & Space Complexity

Time Complexity: $O(n)$

Every element is processed a constant number of times. Although some elements can be swapped more than once, the pointers move monotonically and the total work remains linear.

Space Complexity: $O(1)$

The algorithm sorts the array in-place and uses only three pointer variables.

This is one of the classic examples of achieving both $O(n)$ time and $O(1)$ auxiliary space.

10. ■ DNF vs Other Approaches

Approach 1 — General sorting

Time: $O(n \log n)$

Space: depends on sorting implementation.

Approach 2 — Counting 0, 1 and 2

Time: $O(n)$, usually requires a counting pass followed by rewriting.

Space: $O(1)$ if only three counters are used.

Approach 3 — Dutch National Flag

Time: $O(n)$

Space: $O(1)$

Pass: one main scan

In-place: Yes

DNF is especially elegant when the problem explicitly asks for a one-pass or in-place solution.

11. ■■■ Common Mistakes

1. Incrementing mid after handling a 2.

Why wrong? The swapped-in value from high is still unprocessed.

2. Using while (mid < high) instead of while (mid <= high).

The last unknown element can be skipped.

3. Forgetting to increment low when a 0 is placed.

This breaks the 0-region invariant.

4. Treating DNF as a general sorting algorithm.

It is designed for three categories (or, more generally, three partitions), not arbitrary values without modification.

12. ■ Why does it work?

The algorithm maintains a strict invariant:

- Everything before low is 0.
- Everything from low to mid-1 is 1.
- Everything from high+1 onward is 2.
- Only mid to high remains unknown.

Each operation preserves these regions:

0 → goes to the left region.

1 → remains in the middle region.

2 → goes to the right region.

Therefore, when the unknown region becomes empty, the complete array is partitioned correctly.

13. ■ LeetCode Problem 1

LeetCode 75 — Sort Colors

Problem: Given an array containing 0, 1 and 2, sort it in-place.

Pattern: Dutch National Flag

Key observation:

The problem directly matches the three-region invariant.

Target complexity:

$O(n)$ time and $O(1)$ extra space.

14. ■ LeetCode Problems 2–3

LeetCode 2161 — Partition Array According to Given Pivot

Use the same partitioning mindset: separate elements into regions around a pivot.

LeetCode 905 — Sort Array By Parity

A two-way partitioning problem. The same pointer/invariant thinking helps: maintain a processed region and an unprocessed region.

These problems help you generalize the idea beyond exactly 0, 1 and 2.

15. ■ Interview Tip

If an interviewer asks you to sort an array containing only 0, 1 and 2, do not immediately reach for `std::sort`.

Say:

"I can solve this using the Dutch National Flag algorithm by maintaining low, mid and high pointers. It gives $O(n)$ time and $O(1)$ extra space."

Then explain why you do not increment mid after swapping a 2.

16. ■ Pattern Recognition

Look for DNF-style partitioning when you see:

- Exactly three categories.
- “Sort in-place.”
- “One pass.”
- $O(n)$ expected time.
- Partition elements into left / middle / right groups.
- A pivot-based three-way partition.

The deeper lesson is not memorizing three pointers—it is maintaining a correct invariant while shrinking the unknown region.

17. ■ Quick Revision Sheet

Algorithm: Dutch National Flag

Pointers: low, mid, high

0: swap(low, mid), low++, mid++

1: mid++

2: swap(mid, high), high--

Loop: while (mid <= high)

Time: $O(n)$

Space: $O(1)$

Main rule: After swapping a 2, re-check mid.

18. ■ What I'll learn tomorrow

Day 19 — Merge Intervals

Tomorrow we move from array partitioning to interval-based problem solving.

We will learn:

- What an interval is
- Why sorting helps
- How overlapping intervals are detected
- Merge logic
- C++ implementation
- Complexity
- Common interview patterns
- LeetCode practice problems